Schritt 38 — Custom Scalars in GraphQL-Ruby

Die wichtigste Frage zuerst:

> Gehört ein Custom Scalar zu GraphQL oder zu Rails? Sendet der Client einen Scalar im Request?

Scalar ist ein Konzept von GraphQL. In Rails/graphql-ruby definierst du, wie vom Client gesendete Daten geparst und Ruby-Daten für die Response serialisiert werden.

Der Client sendet keine „Ruby-Klasse". Er sendet nur einen Wert, der zum GraphQL-Scalar-Typ passt.

Beispiel:

```
dueDate: ISO8601Date
```

Der Client sendet:

```
{
  "dueDate": "2026-08-25"
}
```

Rails/graphql-ruby parst diesen String in einen passenden Ruby-Wert.

1. Was ist ein Scalar?

GraphQL besitzt grundlegende Scalars:

```
String
Int
Float
Boolean
ID
```

Beispiel:

```
field :id,
      ID,
      null: false

field :title,
      String,
      null: false

field :completed,
      Boolean,
      null: false
```

Rails besitzt jedoch viele weitere Typen:

```
Date
DateTime
Time
Decimal
JSON
Money
UUID
```

Dann benötigt man einen passenden GraphQL Scalar.

2. Wo befindet sich der Scalar im System?

Flow:

```
CLIENT
|
| JSON value
|
▼
GraphQL Scalar
|
| parse / validate
|
▼
Ruby value
|
▼
Resolver / Mutation
|
▼
Model / Service
```

Beim Zurückgeben von Daten:

```
Ruby value
|
▼
GraphQL Scalar
|
| serialize
|
▼
JSON response
```

3. Praxisbeispiel: due_date

Im Rails Model:

```
class Task < ApplicationRecord
  # due_date :date
end
```

In GraphQL:

```
field :due_date,
      GraphQL::Types::ISO8601Date,
      null: true
```

Client Query:

```
query GetTask($id: ID!) {
  task(id: $id) {
    id
    title
    dueDate
  }
}
```

Response:

```
{
  "data": {
    "task": {
      "id": "10",
      "title": "GraphQL",
      "dueDate": "2026-08-25"
    }
  }
}
```

4. ISO8601Date gehört zu GraphQL oder Rails?

Date ist ein Ruby/Rails-Typ.

ISO8601Date ist hier ein von graphql-ruby bereitgestellter GraphQL Scalar.

Rails:

```
due_date
# Date object
```

GraphQL:

```
dueDate: ISO8601Date
```

Client:

```
"2026-08-25"
```

Drei unterschiedliche Ebenen.

5. Scalar im Input

Ví dụ TaskInput:

```
module Types
  module Inputs
    class TaskInput < Types::BaseInputObject
      argument :title,
               String,
               required: true

      argument :due_date,
               GraphQL::Types::ISO8601Date,
               required: false
```

```
    end
  end
end
```

Client mutation:

```
mutation CreateTask(
  $input: CreateTaskInput!
) {
  createTask(input: $input) {
    task {
      id
      title
      dueDate
    }

    errors
  }
}
```

Variables:

```
{
  "input": {
    "projectId": "10",
    "attributes": {
      "title": "Finish GraphQL",
      "dueDate": "2026-08-25"
    }
  }
}
```

6. Was erhält der Resolver?

Der Client sendet:

```
"dueDate": "2026-08-25"
```

graphql-ruby parst den Scalar.

Die Mutation erhält ungefähr:

```
attributes[:due_date]
# => Date object
```

Nicht:

```
"2026-08-25"
```

wenn der Scalar bereits in Date geparst wurde.

Das ist ein großer Vorteil von Scalars.

7. Date und DateTime sind unterschiedlich

Unser Projekt sollte verwenden:

```
due_date
→ Date

created_at
→ DateTime

updated_at
→ DateTime
```

GraphQL:

```
field :due_date,
      GraphQL::Types::ISO8601Date,
      null: true

field :created_at,
      GraphQL::Types::ISO8601DateTime,
      null: false

field :updated_at,
      GraphQL::Types::ISO8601DateTime,
      null: false
```

Response:

```
{
```

```
  "dueDate": "2026-08-25",
  "createdAt": "2026-08-17T19:30:00Z"
}
```

8. Warum nicht für alles String verwenden?

Man könnte schreiben:

```
field :due_date,
      String,
      null: true
```

aber das ist nicht empfehlenswert.

Denn das GraphQL-Schema verliert Bedeutung:

```
String
→ beliebiger String

ISO8601Date
→ dies ist ein gültiges Datum
```

Client-Tools verstehen das Schema ebenfalls besser.

9. JSON Scalar

graphql-ruby bietet:

```
GraphQL::Types::JSON
```

Beispiel: Das Model hat:

```
metadata :jsonb
```

GraphQL:

```
field :metadata,
      GraphQL::Types::JSON,
      null: true
```

Response:

```
{
  "metadata": {
    "source": "import",
    "externalId": "ABC123",
    "flags": {
      "urgent": true
    }
  }
}
```

10. Wann JSON verwenden?

Geeignet, wenn die Daten wirklich:

```
dynamic
unstructured
kein festes Schema haben
```

Beispiel:

```
external provider metadata
integration payload
custom settings
```

JSON sollte nicht nur aus Bequemlichkeit verwendet werden.

11. JSON nicht als Ersatz für ObjectType verwenden

Nicht empfohlen:

```
field :project,
      GraphQL::Types::JSON,
      null: false
```

und dann zurückgeben:

```
{
  "id": 10,
  "name": "CRM",
  "owner": {},
  "tasks": []
```

```
}
```

Denn dadurch verlierst du:

```
type safety
validation
introspection
autocomplete
code generation
field authorization
```

Besser:

```
field :project,
      Types::ProjectType,
      null: false
```

12. Custom Was ist ein Scalar?

Ein Custom Scalar ist ein Scalar, den die Anwendung selbst definiert.

Beispiel: Die Business-Domain hat den Typ:

```
Money
```

Man möchte nicht willkürlich verwenden:

```
String
Float
```

Gewünscht ist:

```
Client
"1250.50"
↓
Money Scalar
↓
BigDecimal
```

13. MoneyScalar erstellen

Datei erstellen:

```
app/graphql/types/scalars/money_scalar.rb
```

Code:

```
# frozen_string_literal: true

module Types
  module Scalars
    class MoneyScalar < Types::BaseScalar
      graphql_name "Money"

      description "Monetary decimal value"

      def self.coerce_input(
        value,
        _context
      )
        BigDecimal(
          value.to_s
        )
      rescue ArgumentError
        raise GraphQL::CoercionError,
              "#{value.inspect} is not a valid money value"
      end

      def self.coerce_result(
        value,
        _context
      )
        value
          .to_d
          .to_s("F")
      end
    end
  end
end
```

14. Zwei wichtige Methoden

Ein Custom Scalar hat typischerweise zwei Hauptrichtungen:

```
coerce_input
→ Client → Ruby

coerce_result
→ Ruby → Client
```

Flow:

```
CLIENT
"1250.50"
    |
    ▼
coerce_input
    |
    ▼
BigDecimal
```

Umgekehrt:

```
BigDecimal
    |
    ▼
coerce_result
    |
    ▼
"1250.50"
```

15. coerce_input

Der Client sendet:

```
{
  "budget": "1250.50"
}
```

Scalar:

```
def self.coerce_input(
  value,
  _context
)
  BigDecimal(
    value.to_s
  )
end
```

Der Resolver erhält:

```
BigDecimal("1250.50")
```

16. Ungültiger Input

Wenn der Client sendet:

```
{
  "budget": "abc"
}
```

Der Scalar wird:

```
raise GraphQL::CoercionError
```

GraphQL weist den Request zurück, bevor die Business-Logik ausgeführt wird.

Das ist sehr nützlich.

17. coerce_result

Die Datenbank bzw. das Model hat:

```
project.budget
# BigDecimal
```

GraphQL ruft auf:

```
coerce_result(...)
```

und gibt zurück:

```
{
  "budget": "1250.50"
}
```

18. MoneyScalar im Type verwenden

Beispiel: Project hat:

```
budget :decimal
```

In:

```
app/graphql/types/project_type.rb
```

hinzufügen:

```
field :budget,
      Types::Scalars::MoneyScalar,
      null: true
```

Der Client sieht im Schema:

```
type Project {
  budget: Money
}
```

19. Dùng Custom Scalar im Input

Beispiel UpdateProject:

```
argument :budget,
         Types::Scalars::MoneyScalar,
         required: false
```

Client:

```
mutation UpdateProject(
  $input: UpdateProjectInput!
) {
  updateProject(input: $input) {
    project {
      id
      name
      budget
    }

    errors
  }
}
```

Variables:

```
{
  "input": {
    "id": "10",
    "budget": "2500.75"
  }
}
```

20. Client Axios Request

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      mutation UpdateProject(
        $input: UpdateProjectInput!
      ) {
        updateProject(
          input: $input
        ) {
          project {
            id
            name
            budget
          }

          errors
        }
      }
    `,

    variables: {
      input: {
        id: "10",
        budget: "2500.75"
```

```
      }
    },

    operationName: "UpdateProject"
  },
  {
    withCredentials: true
  }
);
```

Der Client sendet nur:

```
"2500.75"
```

Er muss nicht wissen, was

```
BigDecimal
```

ist.

21. Ist ein Scalar Business-Validation?

Nein.

Ein Scalar sollte behandeln:

```
format
type
parse
serialize
```

Beispiel:

```
"abc"
→ ist kein Money-Wert
```

Der Scalar weist ihn zurück.

Business-Regeln wie:

```
budget >= 0
budget <= 10_000_000
```

gehören dagegen zu:

```
Model
Service
Validation
```

22. Beispiel Model Validation

```
validates :budget,
          numericality: {
            greater_than_or_equal_to: 0
          }
```

Unterscheidung:

```
MoneyScalar
→ ist es ein gültiger Money-Wert?

Project model/service
→ ist dieser Money-Wert für die Business-Regeln gültig?
```

23. Scalar übernimmt keine Authorization

Nicht schreiben:

```
def self.coerce_input(
  value,
  context
)
  unless context[:current_user].admin?
    raise ...
  end
end
```

Das ist nicht seine Verantwortung.

Authorization gehört weiterhin zu:

```
Policy
Resolver
Mutation
```

```
Service
```

Der Scalar verarbeitet nur den Datentyp.

24. Enum ist kein Scalar

Beispiel Task Status:

```
TODO
IN_PROGRESS
DONE
```

Nicht erstellen:

```
scalar TaskStatus
```

Besser:

```
enum TaskStatus {
  TODO
  IN_PROGRESS
  DONE
}
```

Rails:

```
class TaskStatusEnum < Types::BaseEnum
  value "TODO",
        value: "todo"

  value "IN_PROGRESS",
        value: "in_progress"

  value "DONE",
        value: "done"
end
```

25. Input Object ist kein Scalar

Beispiel:

```
filter: {
  status: [TODO]
  priority: [HIGH]
  overdue: true
}
```

Das ist ein Objekt mit mehreren Fields.

Dafür sollte verwendet werden:

```
TaskFilterInput
```

Nicht ein Custom Scalar.

26. Regel zur Auswahl des Typs

```
Ein atomarer Wert
→ Scalar

Eine feste Menge von Werten
→ Enum

Ein Objekt mit mehreren Fields
→ InputObject

Ein zurückgegebenes Objekt
→ ObjectType
```

Beispiel:

```
Date
→ Scalar

TaskStatus
→ Enum

TaskFilter
→ InputObject

Task
→ ObjectType
```

27. Befindet sich ein Custom Scalar im Client Request?

Ja, aber der Client sendet nur den Wert.

Beispiel Schema:

```
budget: Money
```

Der Client sendet:

```
{
  "budget": "2500.75"
}
```

Der Client sendet nicht:

```
MoneyScalar
BigDecimal
Ruby class
```

Das GraphQL-Schema weiß, dass dieses Field den Typ Money hat.

28. Scalar + Variables

Beispiel:

```
mutation UpdateBudget(
  $budget: Money!
) {
  updateBudget(
    budget: $budget
  ) {
    project {
      id
      budget
    }
  }
}
```

Variables:

```
{
  "budget": "5000.00"
}
```

Flow:

```
$budget
↓
Money
↓
coerce_input
↓
BigDecimal
↓
Resolver
```

29. Scalar + Input Object

Praxisnäher:

```
input ProjectUpdateInput {
  budget: Money
}
```

Mutation:

```
mutation UpdateProject(
  $input: UpdateProjectInput!
) {
  updateProject(input: $input) {
    project {
      budget
    }
  }
}
```

Variables:

```
{
  "input": {
    "id": "10",
    "budget": "5000.00"
```

```
  }
}
```

30. Scalar in der Response

GraphQL query:

```
query GetProject($id: ID!) {
  project(id: $id) {
    id
    name
    budget
  }
}
```

Rails hat:

```
BigDecimal("5000.00")
```

Der Client erhält:

```
{
  "data": {
    "project": {
      "id": "10",
      "name": "CRM",
      "budget": "5000.00"
    }
  }
}
```

31. Custom Scalar testen

Tạo:

```
spec/graphql/types/scalars/money_scalar_spec.rb
```

Beispiel:

```
RSpec.describe Types::Scalars::MoneyScalar do
  describe ".coerce_input" do
    it "converts string to BigDecimal" do
      result =
        described_class.coerce_input(
          "1250.50",
          nil
        )

      expect(result)
        .to eq(
          BigDecimal("1250.50")
        )
    end

    it "rejects invalid value" do
      expect do
        described_class.coerce_input(
          "abc",
          nil
        )
      end.to raise_error(
        GraphQL::CoercionError
      )
    end
  end
end
```

32. TaskType của project

Eine klare Version:

```
module Types
  class TaskType < Types::BaseObject
    field :id,
          ID,
          null: false

    field :title,
          String,
          null: false
```

```
    field :description,
          String,
          null: true

    field :status,
          Types::Enums::TaskStatusEnum,
          null: false

    field :priority,
          Types::Enums::TaskPriorityEnum,
          null: false

    field :due_date,
          GraphQL::Types::ISO8601Date,
          null: true

    field :created_at,
          GraphQL::Types::ISO8601DateTime,
          null: false

    field :updated_at,
          GraphQL::Types::ISO8601DateTime,
          null: false

    field :assignee,
          Types::UserType,
          null: true
  end
end
```

Das Client-Schema sieht ungefähr so aus:

```
type Task {
  id: ID!
  title: String!
  description: String
  status: TaskStatusEnum!
  priority: TaskPriorityEnum!
  dueDate: ISO8601Date
  createdAt: ISO8601DateTime!
  updatedAt: ISO8601DateTime!
  assignee: User
}
```

33. Braucht das aktuelle Projekt wirklich einen Custom Scalar?

Aktuell:

```
ID
String
Boolean
Int
ISO8601Date
ISO8601DateTime
```

reicht für den größten Teil des Projekts aus.

Man sollte keinen Custom Scalar nur erstellen, um

```
"trông chuyên nghiệp"
```

Ein Custom Scalar sollte nur existieren, wenn die Business-Domain wirklich einen eigenen atomaren Datentyp benötigt.

34. Beispiele für sinnvolle Custom Scalars

Später könnten sinnvoll sein:

```
Money
UUID
EmailAddress
URL
PhoneNumber
Duration
```

Dabei sollte man prüfen:

```
Braucht man wirklich einen eigenen Typ?
```

```
Oder reichen String + Validation?

Ist spezielles Parsing/Serialisieren nötig?
```

35. Nicht jede Validation in einen Scalar verwandeln

Beispiel E-Mail.

Man könnte erstellen:

```
EmailScalar
```

aber das ist nicht immer nötig.

Wenn E-Mail nur ein Business-Field von User ist:

```
validates :email,
          presence: true
```

und Devise E-Mail bereits behandelt, kann ein eigener Scalar das Schema unnötig komplizierter machen.

36. Muss der Client die Custom-Scalar-Implementierung kennen?

Nein.

Das Frontend muss nur den Contract kennen:

```
budget: Money
```

und das Format, das der Client senden muss:

```
"1250.50"
```

Die Backend-Implementierung:

```
BigDecimal
```

ist ein Server-Detail.

37. Scalar + Fragment / Alias / Directive

Kann normal kombiniert werden:

```
fragment ProjectFields on Project {
  id
  name
  budget
}

query GetProject(
  $id: ID!
  $includeBudget: Boolean!
) {
  currentProject: project(
    id: $id
  ) {
    id
    name

    budget
      @include(if: $includeBudget)
  }
}
```

Hier:

```
currentProject
→ Alias

$includeBudget
→ Variable

@include
→ Directive

budget
→ Money Scalar field
```

38. Diagramm Schritt 38

```
CLIENT
│
│ "2500.75"
│
▼
GraphQL Variable / Input
│
▼
Money Scalar
│
├── coerce_input
│
▼
BigDecimal
│
▼
Resolver / Mutation
│
▼
Policy / Service
│
▼
Model
│
▼
Database
│
▼
BigDecimal
│
▼
Money Scalar
│
├── coerce_result
│
▼
"2500.75"
│
▼
CLIENT
```

Merksätze zu Schritt 38

```
Scalar
→ gehört zu GraphQL

Was macht Rails?
→ Scalar mit graphql-ruby definieren
→ Input parsen
→ Output serialisieren

Was sendet der Client?
→ einen zum Schema passenden Wert

Kennt der Client die Ruby-Klasse?
→ nein

Date
→ ISO8601Date

DateTime
→ ISO8601DateTime

JSON
→ nur für wirklich dynamische Daten verwenden

Enum
→ nein ph□i Scalar

InputObject
→ nein ph□i Scalar

Wann Custom Scalar verwenden?
→ atomarer Datentyp
→ eigene Parse-/Serialize-Regeln
```

Der wichtigste Punkt:

```
Client value
↓
GraphQL Scalar
```

```
↓
Ruby value
```

und umgekehrt:

```
Ruby value
↓
GraphQL Scalar
↓
JSON value
```

Schritt 38 ist hauptsächlich Schema-/Server-Side-Wissen in GraphQL-Ruby, im Unterschied zu Schritt 35–37, die stärker darauf fokussiert sind, wie der Client das GraphQL-Dokument schreibt.